ABRA — System Architecture and Engineering Standards

Version 1.2 · 2026-07-24 · Will Hooper

A review of the whole system, the rules that follow from it, and how those rules are enforced. This document is deliberately blunt. Every fault named here is one this project actually shipped, and each is paired with the standard that prevents its recurrence and the check that enforces it.

---

1. The review, without softening

The models are careful. **The plumbing was not.** Every serious defect found on 2026-07-24 was an infrastructure fault, not a modelling error — and each one silently corrupted a model that was otherwise sound. In order of severity:

1.1 Three implementations of the same rules. Damage, stat stages, spread and mega handling were implemented in `CHOMP/engine/champ-model.js`, `ABRA/engine/medicham2-browser.js`, and a third embedded copy in `ABRA/web/index.html`. Fowler's *Rule of Three* says the third duplication is the moment to act; we were already past it. The cost was realised: when the canonical engine was taught real mega base stats, the rollout engine was left behind and the two disagreed by **30%** on Charizard-Mega-Y's Special Attack, with nothing failing.

1.2 Two hand-maintained copies of the same table, from different sources. Mega abilities existed in `champ-model.js` (from Showdown) and in `medicham2-browser.js` (from Serebii). They happened to agree. Nothing guaranteed it, and nothing would have reported it if they had not.

1.3 A data file that stored only derived values. `data/engine-data.js` held computed level-50 stat lines but no base stats. Derived-only data **cannot be recomputed** — so when the forme rules changed, the browser engine could not follow. It could only copy a number that was already wrong.

1.4 A fix applied to the wrong artifact. 67 mega formes were merged into `engine-data.js` and reported as "in the engine dex". The damage engine reads a *different* file (`champions-damage-lab.html`, parsed with `eval()`), so the fix reached nothing. The claim was made in good faith and was false.

1.5 A parser change that silently destroyed an eval set. Recording mega evolution added the mega forme to `brought`, so a Pokémon that mega-evolved counted twice. `brought` became 5 in ~4,700 games, which failed CHOMP-EV's "exactly four" filter and collapsed its eval set from ~1,200 games to **43**. No test guarded the shape of the store.

1.6 Half the store was duplicate rows. 14,361 lines, 7,315 unique games — 7,040 duplicate lines from divergent reconciliations of the append-only store. *(This document first named `merge -X ours` as the mechanism. CHANGELOG 3.1.2 withdrew that diagnosis the same day in favour of the `merge=union` driver, and CHANGELOG 3.23.0 then recorded a fourth duplication AFTER that driver was removed — so the mechanism is a hypothesis with a known counter-example, not a confirmed cause; corrected here 2026-09-09.)* Every "14,355 games" claim was inflated ~2×, and duplicated rows narrow confidence intervals without adding information. Two marginal results lost significance when it was fixed.

1.7 Cross-boundary identifiers were not normalised. `"sand stream"` in one engine, `"sandstream"` in the other. Both internally consistent; a raw string compare between them is a latent bug.

1.8 Hand-typed constants presented as findings. Role weights (0.6, 0.4) and a belief boost (3.0) were invented, not measured. When measured, the boost was 1.26 and the reported gain shrank.

1.9 A lookup table that was silently incomplete. The nature table held 23 of the 25 natures. Naughty and Lax were simply absent, and an absent nature falls through to the neutral multiplier — so those sets computed with 1.0 where the game applies 1.1 and 0.9. Nothing crashed and no count looked wrong, because **a missing row and a legitimately neutral row are indistinguishable by count**. The earlier check asked "did any nature produce an unexpected multiplier?" and 1.0 is an expected multiplier, so it passed. A partial table is the same class of fault as a duplicated one: knowledge that is asserted rather than enumerated against its source.

The pattern is singular: **knowledge with more than one home, and no mechanism that notices when the homes disagree** — or, in 1.9, knowledge with one home that was never checked against the source it claims to represent.

---

2. The standards

S1 — Single source of truth, and generate the rest

> *"Every piece of knowledge must have a single, unambiguous, authoritative representation within a system." — Hunt & Thomas, The Pragmatic Programmer (1999)*

Where multiple representations are unavoidable (a browser copy, a cached table), **one is definitive and every other is generated by a script**, never hand-synchronised. A generated file carries a header saying so and naming its generator.

Applied: `data/mega-formes.json`, `data/move-effects.json`, `data/battle-formes.json`, `data/species-abilities.json` and the `MC` object in `data/engine-data.js` are all generated.

S2 — Duplication that cannot be removed must be observable

Where a second implementation is genuinely required (the site is buildless and cannot `require()`), a **consumer-driven contract test** asserts that all implementations agree on the rules. This is the Pact pattern: a shared, executable statement of the agreement, run in CI, so divergence fails a build instead of corrupting a model.

Applied: `CHOMP/tests/test-engine-contract.js` — 20 assertions. It caught the mega drift on its first run.

S3 — Store source, not just derived values

Any dataset a consumer might need to recompute from must carry its **inputs**, not only its outputs. Derived values may be included for convenience; they may never be the only representation.

Applied: `engine-data.js` now carries base stats (`bs`) alongside the level-50 line (`st`).

S4 — Every dataset records its provenance

Source URL, capture date, authority level, and refresh policy. If two sources disagree, the higher-authority source wins and the disagreement is reported, not silently resolved. This follows standard practice for reproducibility in production ML (versioning, provenance, reproducibility).

Applied: `CHOMP/data/PROVENANCE.md`; every generated file carries `source` and `generated` fields.

S5 — Normalise identifiers at every boundary

Any identifier crossing a subsystem boundary is compared and keyed as **lowercase alphanumeric only**. Display forms are for humans and never for lookups.

S6 — Assert the invariant, not the incidental

A test states the claim being made. CHOMP-EV's claim is "bring quality does not separate from a coin" — a statement about an *interval*, not a point estimate. Pinning the point broke the test when the eval set legitimately changed. Assert `CI contains the coin`.

S7 — The store has a shape, and it is tested

Structural invariants are checked on every run: no duplicate ids, `brought ⊆ six`, `lead ⊆ brought`, the winner is one of the two players, and every record carries every field. A parser change that breaks the shape must fail immediately, not months later inside a model.

S8 — Measured, never asserted

No constant that affects a result is typed by hand. If a weight matters, it is estimated from training data and the estimate is reported. If it cannot be estimated, the feature is not shipped.

S9 — Golden-master before refactoring

Before changing a computation, record its outputs; after, compare. Approval testing exists precisely to make behaviour-preserving refactors provable rather than hoped for.

Applied: `engine/validate_damage.js` (31 scenarios vs the Smogon calculator) is run before and after every engine change.

S10 — Enumerate the domain, do not spot-check it

Where a rule has a **closed, known domain** — 25 natures, 18 types, 6 stat stages — the test iterates the whole domain and asserts the expected behaviour for each member, including a count assertion that the reference list is complete. Spot-checking a table cannot detect a missing row, because absence usually degrades to a plausible default rather than an error.

Applied: `test-mega-and-boosts.js` now walks all 25 natures and asserts the **direction** of change for all five stats against a neutral baseline. It found Naughty and Lax on its first run.

S12 — Everything is linked. Nothing is hardcoded.

> *Added 2026-07-25, after a night in which a retracted figure was found standing in five documents, a regulation start date was wrong by a month, and a format id was repeated in four files.*

A value that exists in one place must be **referenced** from everywhere else, never **restated**. This is S1 extended from data to prose and configuration, and it is the standard that most of this project's documentation failures reduce to.

In code. No literal that already lives in a config or a data file. Format ids come from `data/regulations.json`, thresholds from `data/quality-filter.json`, the SP budget from the source that measures it. If a constant must be written down once, it is written down once and imported.

In documents. A figure belongs in the artifact that computes it; every other document **links** to it rather than copying the number. A copied number is a number that will be wrong later, and the copy never learns it was retracted. `n=7,971` survived three retractions in five documents for exactly this reason.

In configuration. One key, one meaning, one home. `regulations.json` said the active regulation started `2026-07` while Smogon's own statistics and the archived replays both put it at `2026-06-17` — a hardcoded date nobody could check because nothing pointed at the evidence.

Enforced by: `tests/test-docs-current.js` — a registry of retracted figures that fails the build if one reappears as fact, plus a check that the living documents track the CHANGELOG version. The retraction registry is the mechanism: **when a number is withdrawn, it goes in the registry, and the build refuses to let it**

come back.

S13 — No hand-maintained state. A file nobody generates is a file that will lie.

> *Added 2026-07-25, after* `data/status.js` *— the file that records whether each model works — was found claiming PORY was "the win" on the same day PORY was shown to tie a two-feature baseline, and claiming MEW was "roadmap" while MEW had 200,004 validated games on disk.*

S12 says a value lives in one place and is referenced. S13 says the harder thing: **if a fact can be derived from an artifact, no human may type it.** A hand-kept file is a promise to remember, and this project has now demonstrated four times that the promise is not kept — a retracted figure in five documents, a regulation date wrong by a month, a site quoting six different dataset sizes, and a status file describing a model that had already been refuted.

The test. For every published claim about the project's own state, ask: *what would have to change for this to become wrong, and would anything notice?* If the answer is "a person would have to remember", it must be generated instead.

In practice.

- Model status is derived from the evaluation artifacts — `build/build_status.js` reads `pory-nn.json`, `chomp-ev.json`, `guru.js` and the self-play store, applies a stated bar, and writes `data/status.js`. Shipping a new result changes the site on the next build.
- Every generated file carries a `GENERATED by ... do not hand-edit` header and the date.
- Where no artifact can answer, the fallback is marked `derived:false` so the site can distinguish **"measured as null"** from **"nobody has measured it"**. Those are different claims and conflating them is how a roadmap item starts looking like a result.
- The bar itself is written into the rule, not applied by eye. CHOMP is null because its confidence interval spans a coin, not because someone judged it unimpressive.

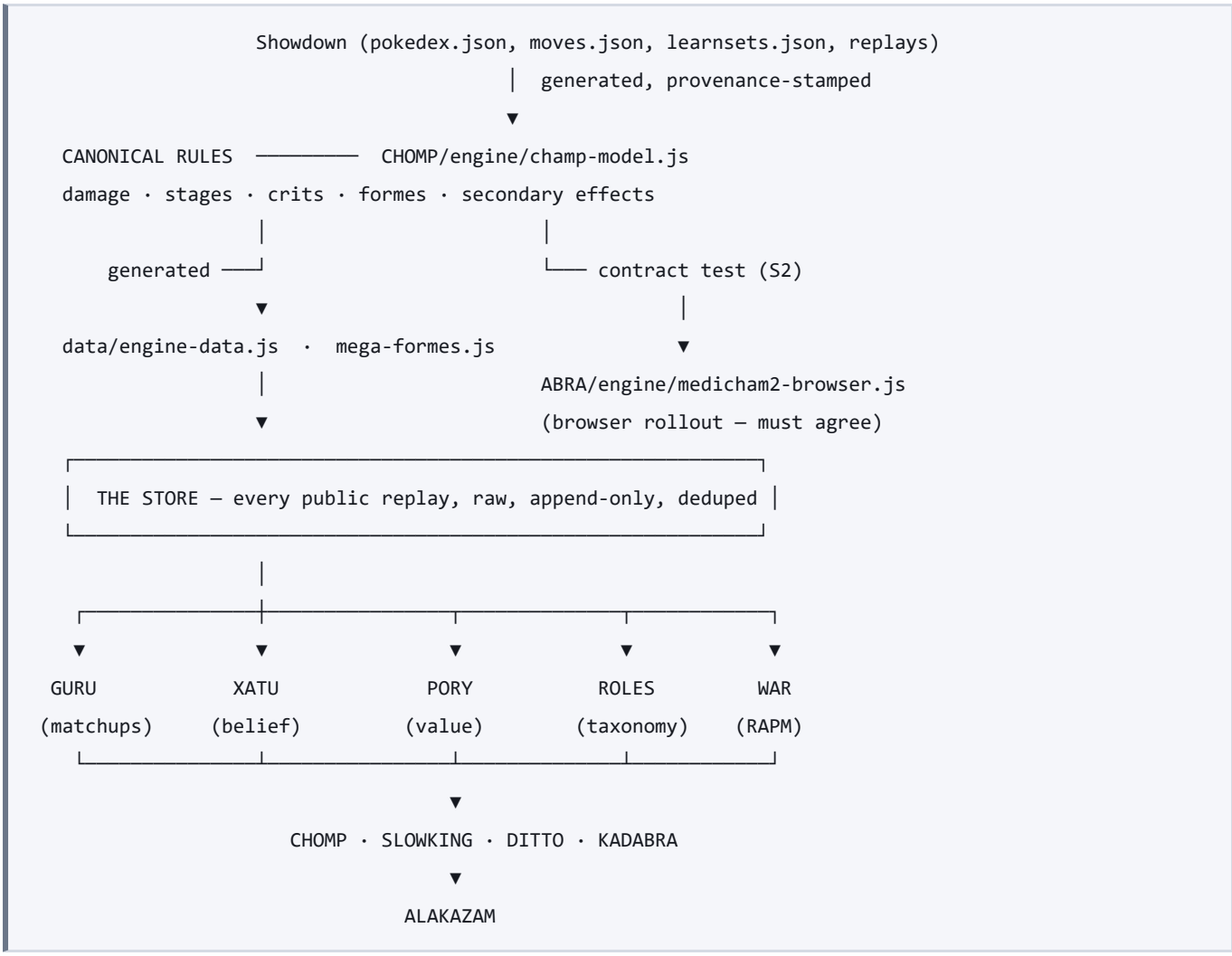
Enforced by: the generator is the only writer; `data/status.js` is regenerated in the same pass as any model change, exactly as S3 requires for docs and CHANGELOG.

S11 — One publisher

Exactly one process commits and pushes. Two publishers racing produced a wedged rebase and hours of lost work. Any automation touching git refuses to act on a repository that is mid-rebase or mid-merge.

---

3. The intended shape



Only one component needs a damage calculator. Everything below it consumes the result. The statistical models (GURU, XATU, PORY, ROLES, WAR) never touch damage at all and are unaffected by engine changes — which is why none of the faults above reached them.

4. Enforcement

Standard	Enforced by	Status
S1 generate, don't synchronise	generator scripts + `generated` / `source` headers	applied
S2 contract test	`CHOMP/tests/test-engine-contract.js`	20 assertions, passing
S3 store source values	`build_engine_data.js` emits `bs`	applied
S4 provenance	`CHOMP/data/PROVENANCE.md` + per-file fields	applied
S5 identifier normalisation	contract test compares normalised	applied
S6 assert the invariant	`sanity_check.py` CHOMP-EV interval test	applied
S7 store shape	`sanity_check.py` §6 + integrity audit	applied
S8 measured not asserted	`test-xatu-belief.py` re-derives the boost	applied
S9 golden master	`validate_damage.js` before/after	applied
S10 enumerate closed domains	`test-mega-and-boosts.js` walks all 25 natures	applied
S11 one publisher	`push-all.bat` guard, mid-rebase refusal	applied

Standard	Enforced by	Status
S12 linked, never hardcoded	`tests/test-docs-current.js` — retraction registry + version currency	applied 2026-07-25

5. Known gaps, stated plainly

- **The canonical engine's dex is still scraped from an HTML file** (`champions-damage-lab.html` , parsed with `eval()`). It should be a real data file with a generator. This is the single largest remaining violation of S1 and is the reason fault 1.4 was possible.
- **The site's embedded engine copy** (`web/index.html`) is not yet covered by the contract test.
- **The rollout engine applies a RANDOM status** (`medicham2-browser.js:205` picks from `['brn','par','slp']` uniformly) and **only Fake Out can flinch** (`:222`). This is fault 1.1 recurring: the shared rulebook was built, tested, and never plugged in, so the rollout kept a second and worse one. Highest-severity open item.
- **Only 25.4% of stored games are clean** — 55.7% involve a bot, 24.0% end in a forfeit, 8.1% run under three turns. There is no shared quality filter, so each model decides for itself.
- **Secondary effects are defined but not yet consumed by the rollout engine** — the rulebook exists and is tested at the point of definition; wiring it into turn simulation is outstanding.
- `brought` **is still 5 in ~115 games** and 0 in 176; not yet explained.

Sources

Hunt & Thomas, *The Pragmatic Programmer* (1999) — DRY, single source of truth · Fowler, *Refactoring* — Rule of Three, duplicated logic · consumer-driven contract testing (Pact) · approval / golden-master testing · versioning, provenance and reproducibility in production ML (Kästner, CMU).